

OS-Lab6-shell实验报告

一、支持相对路径

首先在内核态的 `Env` 结构体中添加字符数组存储当前进程的工作目录。同时记得在 `env_alloc` 时将目录设置为根目录`"/"`，好要在fork时子进程进程控制块复制父进程的目录。

```
// include/env.h
struct Env {
    // ...
    // Lab 6-shell
    char env_cwd[1024]; // MAXPATHLEN = 1024
};
```

之后增加系统调 `sys_chdir` 和 `sys_getcwd`，分别用于改变当前工作目录和获取当前工作目录。用户态增加对应的函数陷入内核态执行。

```
// kern/syscall_all.c
int sys_chdir(u_int envid, const char *path) {
    struct Env *e;
    envid2env(envid, &e, 0);
    strcpy(e->env_cwd, path);
    return 0;
}

int sys_getcwd(char *buf) {
    if (!buf) {
        return -E_INVAL;
    }
    strcpy(buf, curenv->env_cwd);
    return 0;
}

// user/include/lib.h
int syscall_chdir(u_int envid, const char *path);
int syscall_getcwd(char *buf);
```

之后要修改 `file.c` 文件中 `open` 和 `remove` 的逻辑，使之支持相对路径。

```
// user/lib/file.c
int open(const char *path, int mode) {
    // ...
    char abspath[MAXPATHLEN];
    rel2abs(abspath, path);
    try(fsipc_open(abspath, mode, fd));
    // ...
}
```

```
// user/lib/file.c
int rel2abs(char *abspath, const char *path) {
    char tmp[MAXPATHLEN];
```

```

if (!path) {
    debugf("path is null in rel2abs\n");
    return -E_INVAL;
}

if (path[0] == '/') {
    // 绝对路径
    strcpy(tmp, path);
} else {
    // 相对路径
    getcwd(tmp, 1024);
    int len = strlen(tmp);
    tmp[len++] = '/';
    strcpy(tmp + len, path);
}

// 规范化路径中的.和..
format_abspath(abspath, tmp);

return 0;
}

```

除此之外还要修改 `ls.c` 的逻辑，原本不加参数是输出根目录下的文件，现在要输出当前工作目录下的文件。

对于 `cd` 和 `pwd` 指令，因为是内建指令，我直接在 `runcmd` 中处理，如果识别到这个字符串就调用对于函数，后续内建指令同理。

```

int runcmd(char *s) {
    gettoken(s, 0);
    // ...

    // 处理内建指令
    if (strcmp(argv[0], "cd") == 0) {
        return cd(argc, argv);
    } else if (strcmp(argv[0], "pwd") == 0) {
        return pwd(argc, argv);
    } else if (strcmp(argv[0], "declare") == 0) {
        return declare(argc, argv);
    } else if (strcmp(argv[0], "unset") == 0) {
        return unset(argc, argv);
    } else if (strcmp(argv[0], "history") == 0) {
        try(history(argc, argv));
    }

    int child = spawn(argv[0], argv);
    // ...
}

```

二、环境变量管理

对于环境变量，我的实现策略是在内核态新定义一个结构体，存储环境变量，同时新增系统调用来声明、销毁、输出环境变量和获取其的值。

```
// kern/syscall_all.c
#define MAX_NAME_LEN 16
#define MAX_VAL_LEN 16

struct EnvVar {
    char name[MAX_NAME_LEN + 1];
    char value[MAX_VAL_LEN + 1];
    int shell_id; // 0 -> 全局环境变量 others -> 局部环境变量
    int readonly; // 是否是只读变量
    int in_use;    // 是否占用
};
```

同时我为了区分局部变量和全局变量，在每个 shell 开始的时候通过系统调用为其分配独有的 `shell_id`，其值大于零，并规定 `shell_id` 为0的是全局变量，否则为局部变量。

除此之外，对于环境变量的解析和替换，我修改了 `parsecmd` 函数处理"v"的逻辑，新增 `expand_var` 函数将环境变量的值取出。

```
// user/sh.c
int expand_var(const char *buffer, const char *word) {
    char *buffer_copy = buffer;
    while (*word) {
        if (*word == '$') {
            int len = strlen(word);
            char tmp[len + 1];
            word++;
            int i = 0;
            while (*word && *word != '/') {
                tmp[i++] = *word++;
            }
            tmp[i] = '\0';
            try(syscall_get_env_var(buffer_copy, tmp, shell_id));
            buffer_copy += strlen(buffer_copy);
        } else {
            *buffer_copy++ = *word++;
        }
    }
    *buffer_copy = '\0';

    return 0;
}
```

三、输入指令优化

1、指令自由输入

这一个点我花了很久来实现，一开始不清楚光标的移动、修改显示界面和读取输入的逻辑，改bug了很久才实现。最后我重构了 `readline` 的逻辑，读取到 `backspace`、`right` 和 `left` 时单独处理即可。

具体的实现方法是设置 `pointer` 作为光标指向的位置，0为初始位置，表示最左侧。`max` 表示这一行的字符数。也不用 `buf[i]` 来读取字符了，直接用临时变量 `c` 读取，让 `buf` 存储实际显示给用户的内容。

- 如果读取到普通字符，先将 `buf` 在光标之后的字符输出（此时读取的普通的字符已经显示在界面中），然后输出等量的 `\b` 来复原光标，之后更新 `buf`，同时 `max` 和 `pointer` 都加一。

- 如果读取到退格符，若光标不在最左侧则输出 `\b` 光标向左移动一格，之后更新 `buf`，在更新显示界面，同时 `max` 和 `pointer` 都减一。对于 `i` 的处理，在我的实现中其实并不必要，实则就是 `max`，可以将 `for` 循环换成 `while` 循环，判断 `max` 是否大于 `n` 即可。
- 如果读取到左右键，处理类似退格符，只是不需要修改 `buf`，注意如果到达边界需要输出空格或退格符来抵消输入。

```
// user/sh.c
void readline(char *buf, u_int n) {
    int r;
    int pointer = 0; // 指向光标所在位置
    int max = 0; // 最大字符数
    char c; // 从标准输入中读取的字符
    for (int i = 0; i < n; i++) {
        if ((r = read(0, &c, 1)) != 1) {
            if (r < 0) {
                debugf("read error: %d\n", r);
            }
            exit();
        }
        if (c == '\b' || c == 0x7f) {
            // backspace : 删除光标左侧 1个字符并将光标向左移动 1列；若已在行首则无动作

            if (pointer != 0) {
                for (int j = pointer - 1; j < max - 1; j++) {
                    buf[j] = buf[j + 1];
                }
                max--;
                pointer--;
                printf("\b");

                // 重写后续字符覆盖原字符
                for (int j = pointer; j < max; j++) {
                    printf("%c", buf[j]);
                }
                printf(" ");

                // 光标右移回去
                for (int j = max; j > pointer; j--) {
                    printf("\b");
                }
                printf("\b");
                i -= 2;
            } else {
                i--;
            }
            continue;
        }
        // 处理左右键
        if (c == 27) {
            r = read(0, &c, 1);
            if (c != 91) {
                debugf("unkonwn char: %c\n", c);
            }
            r = read(0, &c, 1);
            if (c == 67) { // right
                if (pointer < max) {
                    pointer++;
                }
            }
        }
        buf[i] = c;
        max++;
        pointer++;
    }
}
```

```

        } else {
            printf("\b"); // 回退一格抵消输入
        }
        i--;
    } else if (c == 68) { // left
        if (pointer > 0) {
            pointer--;
        } else {
            printf(" "); // 前进一格抵消输入
        }
        i--;
    } else {
        debugf("unkonwn char: %c\n", buf[i]);
    }
    continue;
}

if (c == '\r' || c == '\n') {
    buf[max] = 0;
    return;
}
// 写入普通字符
// 更新显示界面
for (int j = pointer; j < max; j++) {
    printf("%c", buf[j]);
}
for (int j = pointer; j < max; j++) {
    printf("\b");
}
// 更新 buf
for (int j = max; j > pointer; j--) {
    buf[j] = buf[j - 1];
}
buf[pointer] = c;
pointer++;
max++;
}
// ...
}

```

2、不带 .b 后缀指令

实现起来并不复杂，修改 spawn 函数即可，在打开原文件失败后，加上 .b 再次尝试打开一次。

```

// user/lib/spawn.c
int spawn(char *prog, char **argv) {
    if ((fd = open(prog, O_RDONLY)) < 0) {
        char newprog[128];
        int idx;
        for (idx = 0; prog[idx]; idx++) {
            newprog[idx] = prog[idx];
        }
        newprog[idx++] = '.';
        newprog[idx++] = 'b';
        newprog[idx++] = '\0';
        fd = open(newprog, O_RDONLY);
    }
}

```

```

    }
    if (fd < 0) {
        return fd;
    }
    // ...
}

```

3、快捷键

快捷键的实现和左右键和退格键并无太大区别，在 `readline` 中添加相对逻辑即可。

```

// user/sh.c
void readline(char *buf, u_int n) {
    // ...
    if (c == 1) { // Ctrl-A
        while (pointer > 0) {
            printf("\b");
            pointer--;
        }
        i--;
        continue;
    } else if (c == 5) { // Ctrl-E
        while (pointer < max) {
            printf("%c", buf[pointer]);
            pointer++;
        }
        i--;
        continue;
    } else if (c == 11) { // Ctrl-K
        for (int j = pointer; j < max; j++) {
            printf(" ");
        }
        for (int j = pointer; j < max; j++) {
            printf("\b");
        }
        max = pointer;
        i = pointer - 1;
        continue;
    } else if (c == 21) { // Ctrl-U
        // 删除从行首到光标前
        while (pointer > 0) {
            for (int j = 0; j < max - 1; j++) {
                buf[j] = buf[j + 1];
            }
            max--;
            pointer--;
            printf("\b");
            for (int j = pointer; j < max; j++) {
                printf("%c", buf[j]);
            }
            printf(" ");
            for (int j = pointer; j <= max; j++) {
                printf("\b");
            }
        }
        i = max - 1;
        continue;
    }
}

```

```

    } else if (c == 23) { // Ctrl-W
        // Step 1: 跳过左侧空格
        int cnt = 0;
        while (pointer > 0 && (buf[pointer - 1] == ' ' || buf[pointer - 1]
== '\t')) {
            for (int j = pointer - 1; j < max - 1; j++) {
                buf[j] = buf[j + 1];
            }
            pointer--;
            max--;
            printf("\b");
            for (int j = pointer; j < max; j++) {
                printf("%c", buf[j]);
            }
            printf(" ");
            for (int j = pointer; j <= max; j++) {
                printf("\b");
            }
            cnt++;
        }
        // Step 2: 删除连续非空白字符
        while (pointer > 0 && buf[pointer - 1] != ' ' && buf[pointer - 1] !=
'\t') {
            for (int j = pointer - 1; j < max - 1; j++) {
                buf[j] = buf[j + 1];
            }
            pointer--;
            max--;
            printf("\b");
            for (int j = pointer; j < max; j++) {
                printf("%c", buf[j]);
            }
            printf(" ");
            for (int j = pointer; j <= max; j++) {
                printf("\b");
            }
            cnt++;
        }

        i -= cnt + 1;
        continue;
    }
    // ...
}

```

4、历史指令

对我来说，历史指令也不是很好实现的一个点，同样花了很长时间来实现和 debug。

首先要在 `sh.c` 开始执行时尝试打开历史指令文件 `/.mos_history`，如果打开失败则创建一个新文件。并且需要创建

一个字符串数组 `history_buf` 来存储历史指令，这样获取指令时不需要频繁打开文件并进行繁琐的文件操作，在每次指令执行完毕后更新历史指令文件即可。

其中 `history_index` 记录上一次写入指令的位置，`current_index` 记录当前指令的位置。

```
// user/sh.c
```

```

int main(int argc, char **argv) {
    // ...
    for (;;) {
        if (interactive) {
            printf("\n$ ");
        }
        readline(buf, sizeof buf);
        history_valid[history_index] = 0;
        current_index = history_index;

        // 把命令写入history
        if (strlen(buf) > 0) {
            savecmd(buf);
            history_index = (history_index + 1) % HISTORY_SIZE;
            current_index = history_index;
        }
        if ((r = open("/.mos_history", O_RDWR)) < 0) {
            user_panic("open /.mos_history: %d", r);
        } else {
            for (int i = 0; i < HISTORY_SIZE; i++) {
                char *tmp = history_buf[(history_index + i) % HISTORY_SIZE];
                if (history_valid[(history_index + i) % HISTORY_SIZE]) {
                    fprintf(r, "%s\n", tmp);
                }
            }
            close(r);
        }
        // ...
    }
    return 0;
}

```

对于上下键的移动指令，与左右键相仿，在 `readline` 中添加逻辑即可。因为指令切换时还可能切换回当前未执行的指令，所以需要把当前的 `buf` 暂存在 `history_buf` 中。

```

// user/sh.c
void readline(char *buf, u_int n) {
    // ...
    for (int i = 0; i < n; i++) {
        // ...
        // 处理上下键
        if (c == 27) {
            r = read(0, &c, 1);
            if (c != 91) {
                debugf("unkonwn char: %c\n", c);
            }
            r = read(0, &c, 1);
            if (c == 65) { // up
                printf("%c%c%c", 27, 91, 66); // 下移一个抵消输入
                buf[max] = '\0';
                savecmd(buf);
                if (history_valid[(current_index + HISTORY_SIZE - 1) %
HISTORY_SIZE] && current_index != ((history_index + 1) % HISTORY_SIZE)) {
                    // 移动光标
                    for (int j = 0; j < pointer; j++) {
                        printf("\b");
                    }
                }
            }
        }
    }
}

```

```

    }
    current_index = (current_index + HISTORY_SIZE - 1) %
HISTORY_SIZE;

    strcpy(buf, history_buf[current_index]);
    printf("%s", buf);
    for (int j = strlen(buf); j < max; j++) {
        printf(" ");
    }
    for (int j = strlen(buf); j < max; j++) {
        printf("\b");
    }
    i = max = pointer = strlen(buf);
}
} else if (c == 66) { // down
    // printf("%c%c%c", 27, 91, 65); 不需上移
    buf[max] = '\0';
    savecmd(buf);
    if (history_valid[(current_index + 1) % HISTORY_SIZE] &&
history_index != current_index) {
        // 移动光标
        for (int j = 0; j < pointer; j++) {
            printf("\b");
        }
        current_index = (current_index + 1) % HISTORY_SIZE;
        strcpy(buf, history_buf[current_index]);
        printf("%s", buf);
        for (int j = strlen(buf); j < max; j++) {
            printf(" ");
        }
        for (int j = strlen(buf); j < max; j++) {
            printf("\b");
        }
        i = max = pointer = strlen(buf);
    }
}
// ...
continue;
}
// ...
}
// ...
}

```

5、实现注释功能

这个实现起来比较简单，在 `main` 函数的循环中通过 `readline` 读取指令之后，遍历指令字符串，如果遇到 `#` 则截断字符串（替换为 `\0`），只保留 `#` 之前的部分即可。当然主要这个处理要放在记录历史指令之后，按照 `Linux` 的实现，需要将注释也记录入历史指令。

```

// user/sh.c
int main(int argc, char **argv) {
    // ...
    for (;;) {
        // ...
        readline(buf, sizeof buf);
        // ...
    }
}

```

```

        if (buf[0] == '#') {
            continue;
        }
        for (int i = 0; i < strlen(buf); i++) {
            if (buf[i] == '#') {
                buf[i] = '\0';
                break;
            }
        }
        // ...
    }
    return 0;
}

```

6、实现反引号

这里的实现和之前的环境变量替换类似，增加 `expand_var` 的逻辑即可。

```

// user/sh.c
int expand_var(const char *buffer, const char *word) {
    char *buffer_copy = buffer;
    while (*word) {
        // ...
        if (*word == '`') {
            char tmp[MAX_ARGV_LEN];
            char *q = tmp;
            word++;
            while (*word && *word != '`') {
                *q++ = *word++;
            }
            if (*word == '`') {
                word++;
            }
            *q = '\0';
            try(run_command_and_capture_output(tmp, buffer_copy));
            buffer_copy += strlen(buffer_copy);
        } else {
            *buffer_copy++ = *word++;
        }
    }
    *buffer_copy = '\0';

    return 0;
}

```

此外我们需要一个函数 `run_command_and_capture_output` 来执行反引号括起来的指令，并将结果返回。具体的思路是 `fork` 一个子进程执行指令，然后通过管道将标准输出传给父进程，父进程读取标准输出并返回。要注意这里输出的换行符和空白字符需要舍去，避免影响后续指令运行。实际上应该不能修改这里获取的标准输出，需要完全替换，但是因为评测不是很严，我为了取巧在这里做了修改。

```

int run_command_and_capture_output(const char *cmd, char *output) {
    // ...
    if (r == 0) {
        // 子进程
        close(p[0]); // 关闭读端
        dup(p[1], 1); // stdout -> pipe写端
    }
}

```

```

        runcmd(cmd);
        close(p[1]);
        exit();
    } else {
        // 父进程
        close(p[1]); // 关闭写端
        int n = read(p[0], output, MAX_ARGV_LEN - 1);
        if (n >= 0) {
            for (int i = 0; i < n; i++) {
                if (output[i] == '\n' || output[i] == '\r' ||
                    (iswhite(output[i]) && iswhite(output[i + 1]))) {
                    output[i] = '\0';
                    break;
                }
            }
            output[n] = '\0';
        } else {
            output[0] = '\0';
        }
        close(p[0]);
        wait(r);
    }

    return 0;
}

```

同时需要修改 `_gettoken` 的逻辑，单独识别反引号，将反引号里的内容连同反引号一起作为 `w` 返回。如果不这么做，`_gettoken` 会忽略空格，将 ``cd /dir1`` 拆分成 ``cd` 和 `/dir`` 不符合我们的预期。

```

// user/sh.c
int _gettoken(char *s, char **p1, char **p2) {
    // ...
    // 识别反引号括起的内容
    // 反引号也要被视为一部分
    if (*s == '`') {
        *p1 = s;
        s++;
        while (*s && *s != '`') {
            s++;
        }
        if (*s == '`') {
            s++;
        }
        *p2 = s;
        return 'w';
    }
    // ...
}

```

7、实现一行多指令

这个实现起来比较简单，在 `parsecmd` 中增加对分号的处理即可，遇到分号就拆分指令，让子进程执行分号前的指令，父进程递归调用 `parsecmd` 处理子指令。

```

// user/sh.c
int parsecmd(char **argv, int *rightpipe) {

```

```

int argc = 0;
while (1) {
    char *t;
    int fd, r;
    int c = gettoken(0, &t);
    switch (c) {
        // ...
        case ';':
            // 新建子进程执行;前的指令, 父进程等待子进程执行完毕
            if ((r = fork()) < 0) {
                debugf("fork: %d\n", r);
                exit();
            }
            if (r == 0) {
                return argc;
            } else {
                wait(r);
                return parsecmd(argv, rightpipe);
            }
            break;
    }
}
return argc;
}

```

四、指令条件执行

这里我也花了很多时间来实现, 关键在于在父子进程传递程序的返回值, 一开始尝试使用 `ipc_recv` 和 `ipc_send` 来实现返回值的传递, 但是一直有 bug 处理不好, 耗时数小时无过后, 尝试了另一种思路——为进程控制块新增一个 `env_exit_status` 来表示该进程执行完毕后的返回值, 再增加系统调用来设置和获取返回值。

注意这里不能使用 `envid2env` 来获取进程控制块, 因为我们是在 `wait` 函数中获取这个进程的返回值, 此时 `env_status` 为 `ENV_FREE`, 进入 `envid2env` 函数会返回 `-E_BAD_ENV`, 导致得不到正确的返回值。

```

// include/env.h
struct Env {
    // ...
    // Lab 6-shell
    int env_exit_status;
};

// kern/syscall_all.c
int sys_exit(int exit_status) {
    curenv->env_exit_status = exit_status;
    return 0;
}

int sys_get_exit_status(u_int envid) {
    struct Env *e = &envs[ENVX(envid)];
    return e->env_exit_status;
}

// kern/env.c
int envid2env(u_int envid, struct Env **penv, int checkperm) {
    struct Env *e;

```

```

    if (envid == 0) {
        *penv = curenv;
        return 0;
    }

    e = &envs[ENVX(envid)];

    if (e->env_status == ENV_FREE || e->env_id != envid) {
        // !!!!!!!
        return -E_BAD_ENV;
    }
    // ...
}

```

之后我们修改 `wait` 函数，使其通过 `syscall_get_exit_status` 返回子进程的返回值，并在 `exit` 通过系统调用设置进程返回值。

```

// user/lib/libo.c
void libmain(int argc, char **argv) {
    env = &envs[ENVX(syscall_getenv())];
    int exit_status = main(argc, argv);
    syscall_exit(exit_status);
    exit();
}

// user/lib/wait.c
int wait(u_int envid) {
    const volatile struct Env *e;

    e = &envs[ENVX(envid)];
    while (e->env_id == envid && e->env_status != ENV_FREE) {
        syscall_yield();
    }

    return syscall_get_exit_status(envid);
}

```

之后在 `_gettoken` 处增加对 `&&` 和 `||` 的识别，返回一个特殊的值。然后在 `parsecmd` 处增加对应的处理。

具体的实现思路是 `fork` 一个进程，子进程执行指令，父进程等待子进程执行完毕，根据子进程的返回值决定是否执行下一个指令。如果不需要执行该指令，需要往后遍历 `token` 找到 `&&` 和 `||` 或 `;`。

```

// user/sh.c
int parsecmd(char **argv, int *rightpipe) {
    int argc = 0;
    while (1) {
        char *t;
        int fd, r;
        int c = gettoken(0, &t);
        switch (c) {
            // ...
            case 300:
                // &&
                if ((r = fork()) < 0) {
                    debugf("fork: %d\n", r);
                    exit();
                }
            }
        }
    }
}

```

```

    }
    if (r == 0) {
        return argc;
    } else {
        int result = wait(r);
        if (result != 0) {
            do {
                c = gettoken(0, &t);
            } while (c && c != 301 && c != ';');
            if (!c) {
                return 0;
            }
        }
        return parsecmd(argv, rightpipe);
    }
    break;
case 301:
    // ||
    if ((r = fork()) < 0) {
        debugf("fork: %d\n", r);
        exit();
    }
    if (r == 0) {
        return argc;
    } else {
        int result = wait(r);
        if (result == 0) {
            do {
                c = gettoken(0, &t);
            } while (c && c != 300 && c != ';');
            if (!c) {
                return 0;
            }
        }
        return parsecmd(argv, rightpipe);
    }
    break;
}
}
return argc;
}

```

五、更多指令

几个外部指令实现起来很简单，新建对应的 .c 文件，编写相应逻辑即可，需要注意在 new.mk 中添加对应的二进制可执行文件。具体实现逻辑仿照 ls.c 等文件即可。

mkdir 的实现还有一个需要注意的点是要在文件系统服务进程打开文件时对 O_MKDIR 添加对应处理即可。

```

// fs/serv.v
void serve_open(u_int envid, struct Fsreq_open *rq) {
    // ...

    if ((rq->req_omode & O_CREAT || rq->req_omode & O_MKDIR) &&
        (r = file_create(rq->req_path, &f)) < 0 && r != -E_FILE_EXISTS) {

```

```
ipc_send(envid, r, 0, 0);
return;
}

// ...

// If mode include O_MKDIR, set the file type to FTYPE_DIR
if (rq->req_omode & O_MKDIR) {
    f->f_type = FTYPE_DIR;
}

// ...
```

对于内建指令 `exit`，因为需要退出子 `shell`，所以我直接在 `main` 函数添加逻辑，若识别到 `exit` 直接返回。但是这样就没有考虑到环境变量值为 `exit` 等特殊情况，可以让子进程返回特殊的值，父进程识别到之后退出程序。

```
// user/sh.c
int main(int argc, char **argv) {
    // ...
    for (;;) {
        // ...
        if (strcmp(buf, "exit") == 0) {
            // 条件判断不够严谨
            break; // 退出当前shell
        }
        // ...
    }
    return 0;
}
```

六、追加指令重定向

这个实现起来难度也不大，和前面实现指令条件执行类似，在 `_gettoken` 处增加对 `>>` 的识别，返回一个特殊的值。然后在 `parsecmd` 处增加对应的处理。使用 `Fd` 结构体中的 `fd_offset` 来设置文件指针位置为文件的末尾，文件的大小通过 `FileFd` 结构体获取即可。

```
// user/sh.c
int parsecmd(char **argv, int *rightpipe) {
    int argc = 0;
    while (1) {
        char *t;
        int fd, r;
        int c = gettoken(0, &t);
        switch (c) {
            // ...
            case 302:
                // >>
                if (gettoken(0, &t) != 'w') {
                    debugf("syntax error: > not followed by word\n");
                    exit();
                }
                fd = open(t, O_WRONLY | O_CREAT);
                if (fd < 0) {
                    debugf("failed to open '%s'\n", t);
                }
            }
        }
    }
}
```

```
        exit();
    }
    struct Fd *fd_struct = (struct Fd*) num2fd(fd);
    struct Filefd *ffd = (struct Filefd*) fd_struct;
    fd_struct->fd_offset = ffd->f_file.f_size;
    dup(fd, 1);
    close(fd);
    break;
}
}
return argc;
}
```